



Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών και
Μηχανικών Υπολογιστών (ΣΗΜΜΥ)

8ο εξάμηνο

Ψηφιακά Συστήματα VLSI

2η εργαστηριακή άσκηση

Δημήτρης Βατάλας

(Α.Μ.: 03121093)

Αριστοτέλης Γεώργιος Δρίτσας

(Α.Μ.: 03122127)

Αθήνα, Μάρτιος 2026

Contents

Θέμα 1: Σύγχρονος Πλήρης Αθροιστής (Full Adder - FA) με περιγραφή συμπεριφοράς (Behavioral).	2
Θέμα 2: Σύγχρονος Αθροιστής διάδοσης κρατουμένου των 4 bits με χρήση της τεχνικής Pipeline	6
Θέμα 3: Συστολικός Πολλαπλασιαστής διάδοσης κρατουμένου των 4 bits	15
Θέμα bonus: Πέρασμα κώδικα στην πλακέτα zybo και πειραματισμός με αυτόν	25

Θέμα 1: Σύγχρονος Πλήρης Αθροιστής (Full Adder - FA) με περιγραφή συμπεριφοράς (Behavioral).

Ο κώδικας σε vhdl για την περιγραφή του σύγχρονου Full Adder και της αρχιτεκτονικής του σε behavioral είναι ο εξής:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Sync_FA is
    Port (
        --FA
        a, b, cin: in std_logic;
        cout, sum: out std_logic;

        --FF
        clk: in std_logic
    );
end Sync_FA;

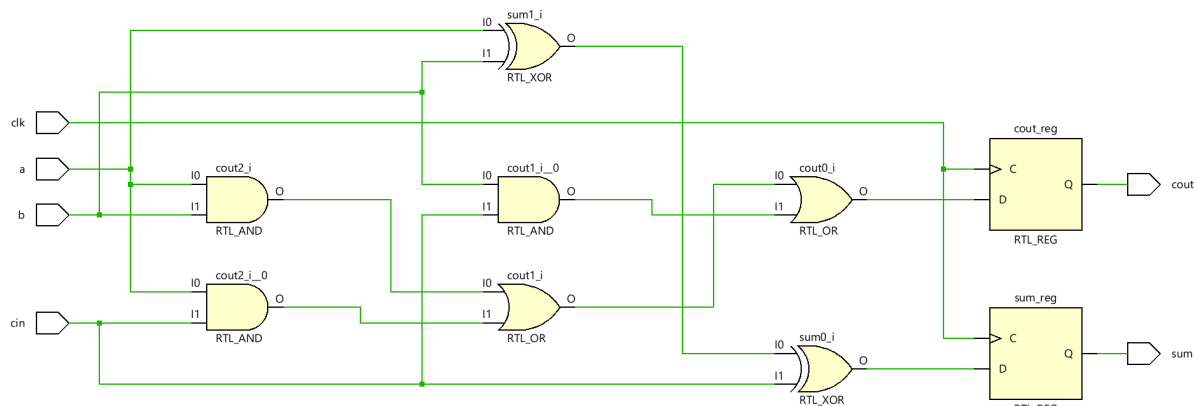
architecture Behavioral of Sync_FA is

begin

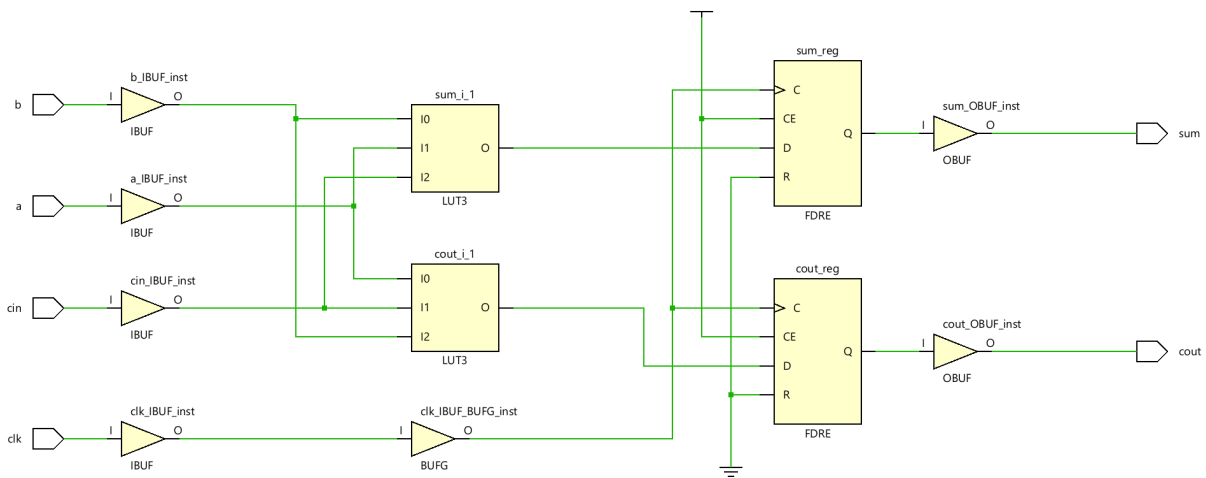
    process(clk)
    begin
        if(rising_edge(clk)) then
            sum <= a xor b xor cin;
            cout <= (a and b) or (a and cin) or (b and cin);
        end if;
    end process;

end Behavioral;
```

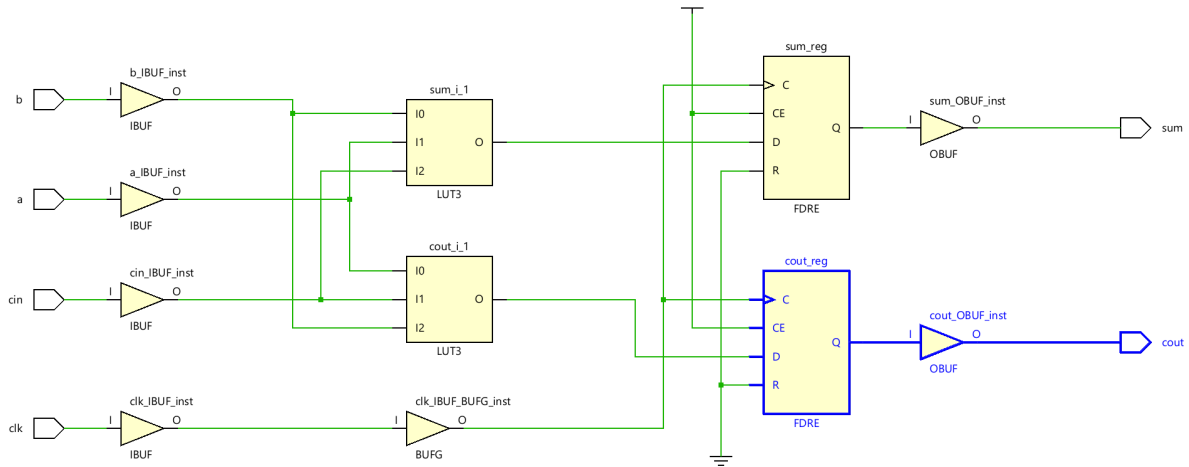
Με τη χρήση του elaborate design παίρνουμε το εξής schematic:



Με τη χρήση του synthesis παίρνουμε το εξής schematic:



Χρησιμοποιώντας το timing summary για το κύκλωμα μας μπορούμε να βρούμε το κρίσιμο μονοπάτι του κυκλώματος μας, το οποίο φαίνεται παρακάτω:



Όπως φαίνεται παρακάτω η χρονική καθυστέρηση του critical path του κυκλώματος είναι 4.076 ns.

Unconstrained Paths - NONE - NONE - Setup														
Name	Slack	Levels	Routes	High Fanout	From	To	Total ...	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
Path 1	∞	2	2	1	cout_reg/C	cout	4.076	3.276	0.800	∞				0.000
Path 2	∞	2	2	1	sum_reg/C	sum	4.076	3.276	0.800	∞				0.000
Path 3	∞	2	3	2	a	sum_reg/D	1.932	1.132	0.800	∞	input port clock			0.000
Path 4	∞	2	3	2	b	cout_reg/D	1.906	1.106	0.800	∞	input port clock			0.000

Ο εντοπισμός του κρίσιμου μονοπατιού είναι σημαντικός, καθώς μπορούμε να παρατηρήσουμε ποιο μονοπάτι δεδομένων είναι το πιο χρονοβόρο στη σχεδίαση μας. Έτσι, αν χρειαστούμε βελτιώσεις στον χρόνο ξέρουμε ποιο μονοπάτι χρειάζεται να αλλάξει. Επιπλέον, η μέγιστη χρονική καθυστέρηση επηρεάζει άμεσα την επιλογή της ελάχιστης συχνότητας ρολογιού.

Για την προσομοίωση και τον έλεγχο της σωστής λειτουργίας του κυκλώματός μας χρησιμοποιούμε το παρακάτω testbench:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Sync_FA_tb is
    -- Port ( );
end Sync_FA_tb;

architecture tb of Sync_FA_tb is

    component Sync_FA is
        Port (
            a, b, cin, clk : in std_logic;
            cout, sum : out std_logic
        );
    end component;

    signal a_tb, b_tb, cin_tb, clk_tb, cout_tb, sum_tb : std_logic := '0';

    constant CLOCK_PERIOD : time := 10 ns;

begin

```

```

DUT : Sync_FA
port map
(
    a => a_tb,
    b => b_tb,
    cin => cin_tb,
    cout => cout_tb,
    sum => sum_tb,
    clk => clk_tb
);

-- Clock generator
CLOCK_GEN : process
begin
    clk_tb <= '0';
    wait for CLOCK_PERIOD / 2;
    clk_tb <= '1';
    wait for CLOCK_PERIOD / 2;
end process;

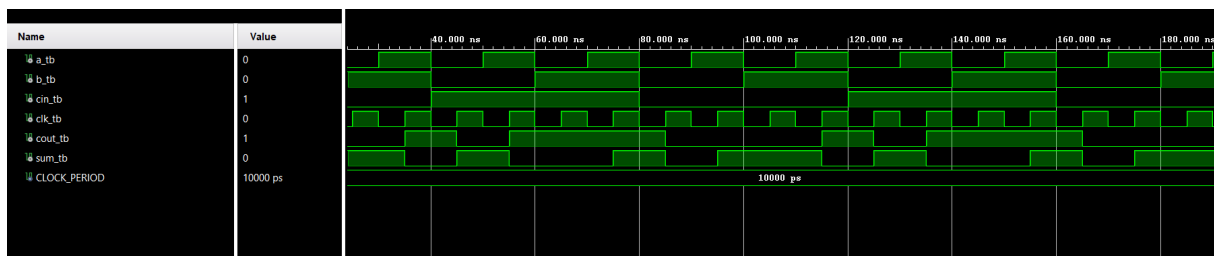
-- Input A
IN_A : process
begin
    a_tb <= '0';
    wait for CLOCK_PERIOD;
    a_tb <= '1';
    wait for CLOCK_PERIOD;
end process;

-- Input B
IN_B : process
begin
    b_tb <= '0';
    wait for CLOCK_PERIOD * 2;
    b_tb <= '1';
    wait for CLOCK_PERIOD * 2;
end process;

-- Carry input
IN_CARRY : process
begin
    cin_tb <= '0';
    wait for CLOCK_PERIOD * 4;
    cin_tb <= '1';
    wait for CLOCK_PERIOD * 4;
end process;

end tb;

```

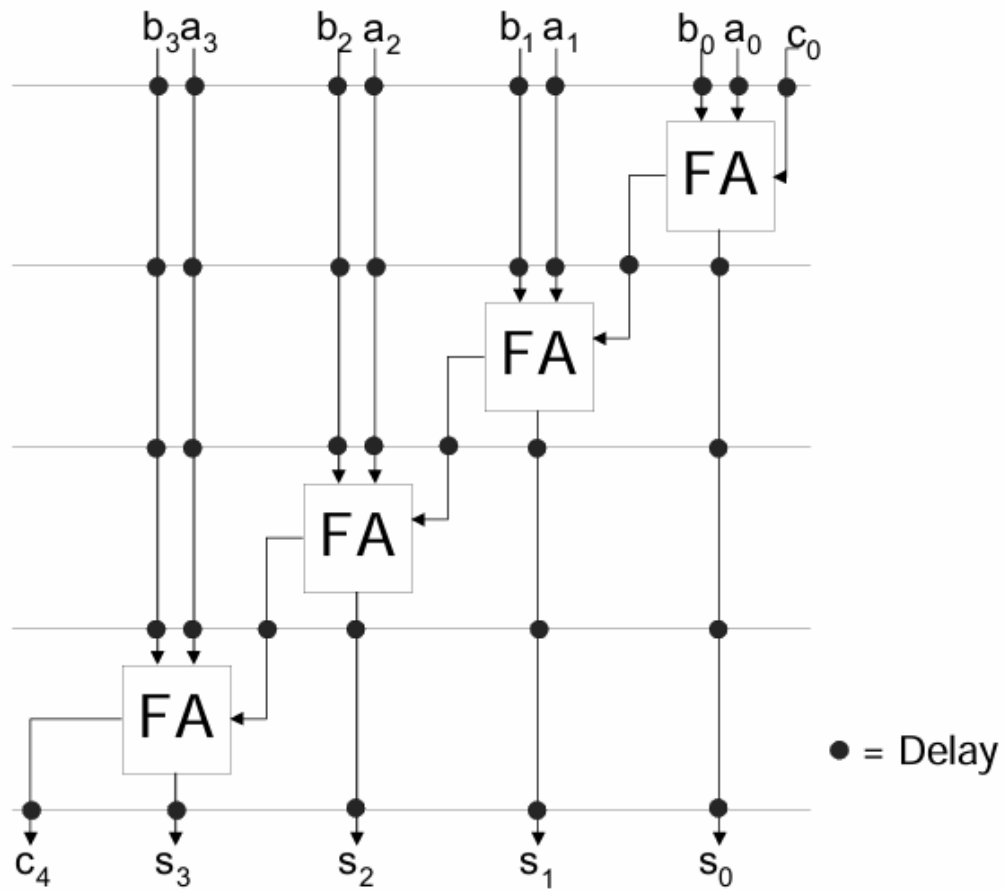


Παρατηρούμε ότι έχουμε ορθά αποτελέσματα για όλους τους συνδυασμούς εισόδων.

Θέμα 2: Σύγχρονος Αθροιστής διάδοσης κρατουμένου των 4 bits με χρήση της τεχνικής Pipeline

Η σχεδίαση του σύγχρονου αθροιστή διάδοσης κρατουμένου 4 bits (Synchronous Carry Propagation Adder), με χρήση πλήρων αθροιστών και τεχνικών pipelining, βασίστηκε στο παρακάτω σχηματικό. Ωστόσο, πραγματοποιήθηκε μία μικρή τροποποίηση: στις εισόδους a_0 και b_0 δεν εισήχθη επιπλέον καθυστέρηση, καθώς αυτή κρίθηκε περιττή. Η επιλογή αυτή συνεπάγεται αντίστοιχες προσαρμογές στα επόμενα στάδια του pipeline, οι οποίες όμως δεν αναλύονται περαιτέρω, δεδομένου ότι πρακτικά αντιστοιχούν στην αφαίρεση ενός κύκλου καθυστέρησης από κάθε ζεύγος εισόδων. Η συγκεκριμένη τροποποίηση δεν επηρεάζει την ορθή λειτουργία του κυκλώματος, καθώς τα bits a_0 και b_0 χρησιμοποιούνται άμεσα στο πρώτο στάδιο του pipeline και, συνεπώς, δεν απαιτείται χρονική ευθυγράμμιση τους με δεδομένα προηγούμενων σταδίων.

Σε γενικότερο πλαίσιο για να απαντήσουμε και ρητά στο ερώτημα της εκφώνησης που αφορά τις καθυστερήσεις που εισάγαμε, εισάγαμε επιπλέον καταχωρητές (καθυστερήσεις): στις εισόδους, μεταξύ των διαδοχικών σταδίων του κυκλώματος αλλά και στις εξόδους, όπως φαίνεται και στο παρακάτω σχεδιάγραμμα, με στόχο την υλοποίηση της pipelined λειτουργίας. Οι καταχωρητές αυτοί εξασφαλίζουν τη χρονική ευθυγράμμιση των δεδομένων, ώστε κάθε στάδιο να λαμβάνει τα κατάλληλα σήματα την σωστή χρονική στιγμή. Με τον τρόπο αυτό διατηρείται η ορθή διάδοση του κρατουμένου και η σωστή λειτουργία του αθροιστή.



Για την υλοποίηση του αθροιστή χρησιμοποιήθηκε ένα module που υλοποιεί έναν shift register με μεταβλητό delay όπως φαίνεται παρακάτω:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity shift_reg is
    generic (
        DELAY : integer := 1 -- number of clock cycles to delay
    );
    Port (
        clk : in  std_logic;
        d   : in  std_logic;
        q   : out std_logic
    );
end shift_reg;

architecture Behavioral of shift_reg is
    signal tmp : std_logic_vector(DELAY-1 downto 0);
begin
    process(clk)
    begin
        if rising_edge(clk) then
            tmp(0) <= d;
        end if;
    end process;
end Behavioral;
```

```

        for i in 1 to DELAY-1 loop
            tmp(i) <= tmp(i-1);
        end loop;
    end if;
end process;

q <= tmp(DELAY-1);
end Behavioral;

```

Ο κώδικας σε vhdl για την περιγραφή του κυκλώματος του αθροιστή και της αρχιτεκτονικής του είναι ο εξής:

```

library IEEE;library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Sync_4FA is
    Port (
        a, b: in  std_logic_vector(3 downto 0);
        cin: in  std_logic;
        sum: out std_logic_vector(3 downto 0);
        cout: out std_logic;
        clk: in  std_logic
    );
end Sync_4FA;

architecture Behavioral of Sync_4FA is

    component Sync_FA is
        Port (
            a, b, cin: in  std_logic;
            sum, cout: out std_logic;
            clk: in  std_logic
        );
    end component;

    component shift_reg is
        generic (DELAY : integer := 1);
        Port (
            clk : in  std_logic;
            d   : in  std_logic;
            q   : out std_logic
        );
    end component;

    signal c : std_logic_vector(4 downto 0);

    signal a_pipe, b_pipe : std_logic_vector(3 downto 0);
    signal sum_pipe : std_logic_vector(3 downto 0);

begin

    c(0) <= cin;

    -- bit 0 (LSB), no delay needed
    a_pipe(0) <= a(0);
    b_pipe(0) <= b(0);

```

```

-- bits 1..3 use shift registers
gen_shift_in: for i in 1 to 3 generate

    SR_a: shift_reg
        generic map (DELAY => i)
        port map (
            clk => clk,
            d   => a(i),
            q   => a_pipe(i)
        );

    SR_b: shift_reg
        generic map (DELAY => i)
        port map (
            clk => clk,
            d   => b(i),
            q   => b_pipe(i)
        );

end generate;

gen_FA: for i in 0 to 3 generate
begin
    FA_i: Sync_FA
        port map(
            a => a_pipe(i),
            b => b_pipe(i),
            cin => c(i),
            sum => sum_pipe(i),
            cout => c(i+1),
            clk => clk
        );
end generate;

-- bit 4 (MSB), no delay needed
sum(3) <= sum_pipe(3);

-- bits 1..3 use shift registers
gen_shift_out: for i in 0 to 2 generate

    SR_sum: shift_reg
        generic map (DELAY => 3 - i)
        port map (
            clk => clk,
            d   => sum_pipe(i),
            q   => sum(i)
        );

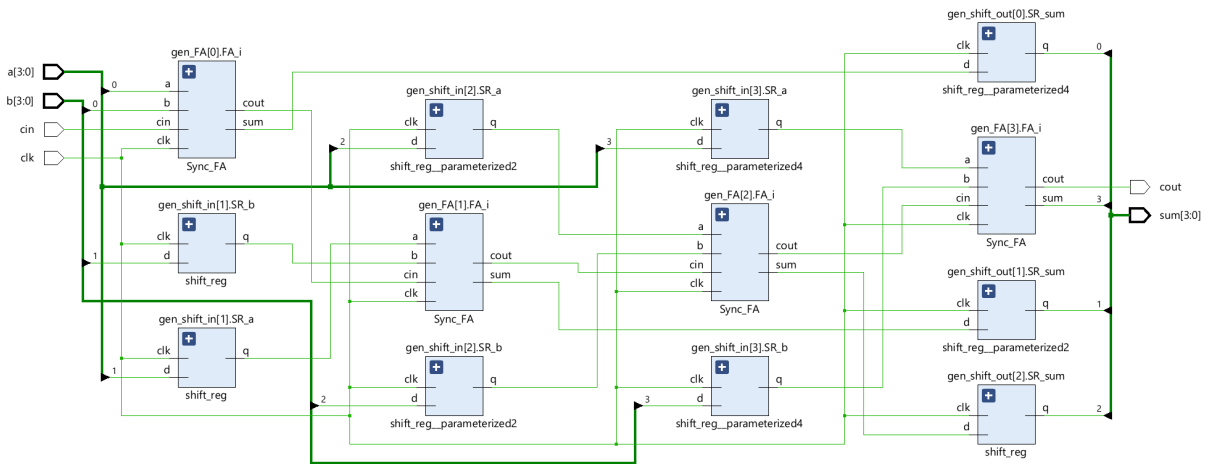
end generate;

cout <= c(4);

end Behavioral;

```

Με τη χρήση του elaborate design παίρνουμε το εξής schematic:



Στο παραπάνω schematic μπορούμε να παρατηρήσουμε τους τέσσερεις full adders, καθώς και τους επιπλέον buffers που χρησιμοποιούνται για τις καθυστερήσεις.

Για την προσομοίωση και τον έλεγχο της σωστής λειτουργίας του κυκλώματός μας χρησιμοποιούμε το παρακάτω testbench:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity Sync_4FA_TB is
-- Port ( );
end Sync_4FA_TB;

architecture tb of Sync_4FA_TB is

component Sync_4FA is
  Port (
    --FA
    a, b: in std_logic_vector(3 downto 0);
    cin: in std_logic;
    sum: out std_logic_vector(3 downto 0);
    cout: out std_logic;

    --FF
    clk: in std_logic
  );
end component;

signal a, b, sum: std_logic_vector(3 downto 0) := "0000";
signal cin, clk, cout: std_logic := '0';

constant CLOCK_PERIOD : time := 10 ns;

begin

  DUT : Sync_4FA
    port map
    (
      a => a,
      b => b,
      cin => cin,
      cout => cout,
      sum => sum,
      clk => clk
    );

  -- Clock generator
  CLOCK_GEN : process
  begin
    clk <= '0';
    wait for CLOCK_PERIOD / 2;
    clk <= '1';
    wait for CLOCK_PERIOD / 2;
  end process;

  -- Input A
  IN_A : process
  begin
```

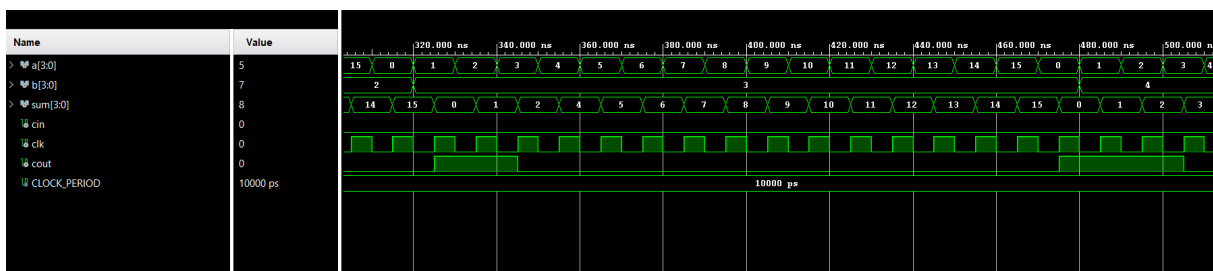
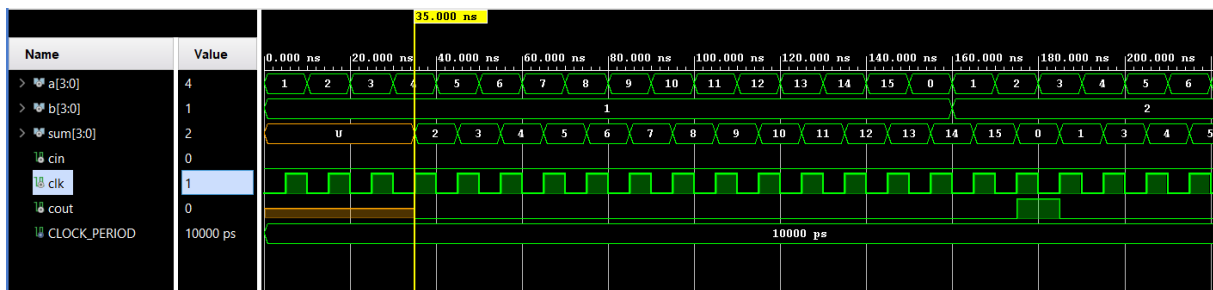
```

        a <= a + 1;
        wait for CLOCK_PERIOD;
    end process;

    -- Input B
    IN_B : process
    begin
        b <= b + 1;
        wait for CLOCK_PERIOD * 16;
    end process;

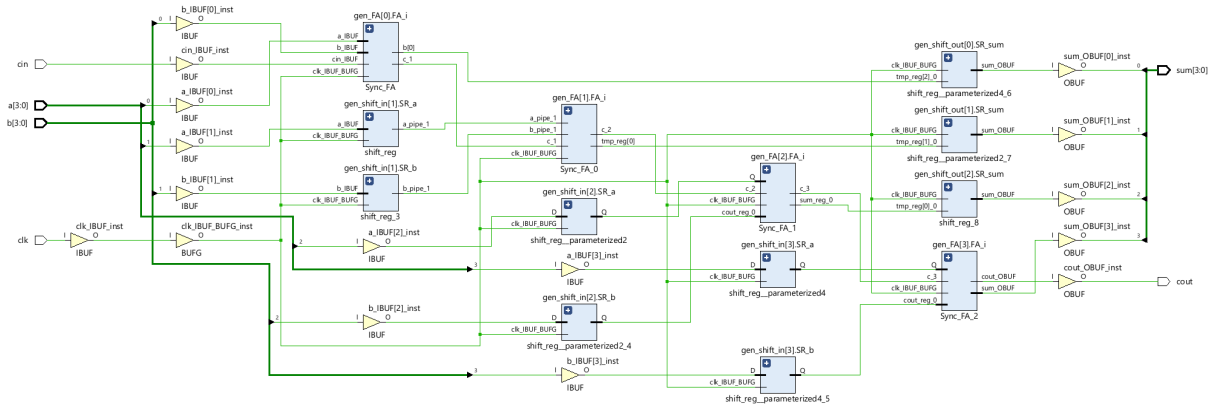
    -- Carry input
    IN_CARRY : process
    begin
        cin <= '0';
        wait for CLOCK_PERIOD * 16 * 16;
        cin <= '1';
        wait for CLOCK_PERIOD * 16 * 16;
    end process;
end tb;

```

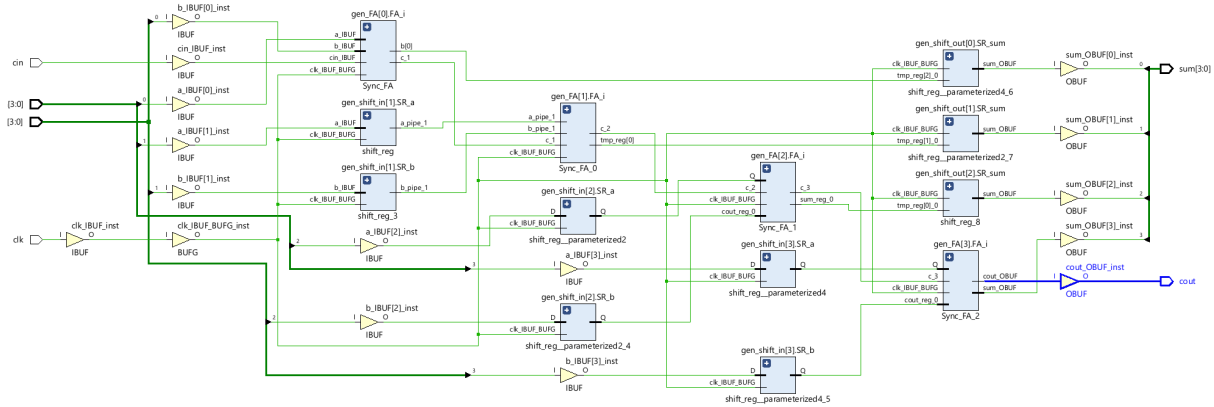


Στην πρώτη εικόνα παρατηρούμε ότι το πρώτο έγκυρο αποτέλεσμα εμφανίζεται στην 4η θετική ακμή του ρολογιού, γεγονός που αντιστοιχεί σε καθυστέρηση 4 κύκλων, όπως αναμένεται λόγω του pipeline που έχει εισαχθεί για λόγους συγχρονισμού. Από τον 4ο κύκλο και έπειτα, το κύκλωμα αποδίδει ένα νέο σωστό αποτέλεσμα σε κάθε κύκλο ρολογιού, όπως επιβεβαιώνεται και από τις δύο εικόνες που παρατίθενται. Συνεπώς, επαληθεύεται η ορθή λειτουργία του κυκλώματος.

Με τη χρήση του synthesis παίρνουμε το εξής schematic:



Χρησιμοποιώντας το timing summary για το κύκλωμα μας μπορούμε να βρούμε το κρίσιμο μονοπάτι του κυκλώματος μας, το οποίο φαίνεται παρακάτω:



Όπως φαίνεται παρακάτω, η χρονική καθυστέρηση του critical path του κυκλώματος είναι 4.076 ns. Αξίζει να σημειωθεί ότι, λόγω της χρήσης pipelining, η καθυστέρηση αυτή αντιστοιχεί σε ένα στάδιο του pipeline και όχι σε μονοπάτι από την είσοδο έως την έξοδο του κυκλώματος. Επιπλέον, παρότι πρόκειται για σύγχρονο κύκλωμα, το slack εμφανίζεται ως άπειρο. Αυτό υποθέτουμε ότι οφείλεται στο γεγονός ότι δεν έχουμε ορίσει κατάλληλα constraints στο design, με αποτέλεσμα το εργαλείο να μην γνωρίζει τις χρονικές απαιτήσεις που πρέπει να ικανοποιηθούν.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
Path 1	∞	2	2	1	gen_FA[3]FA_i/cout_reg/C	cout	4.076	3.276	0.800	∞				0.000
Path 2	∞	2	2	1	gen_shift_out[2].um/tmp_reg[0]/C	sum[2]	4.076	3.276	0.800	∞				0.000
Path 3	∞	2	2	1	gen_FA[3]FA_i/sum_reg/C	sum[3]	4.076	3.276	0.800	∞				0.000
Path 4	∞	2	2	1	gen_shift_out[0].um/tmp_reg[2]/C	sum[0]	4.058	3.258	0.800	∞				0.000
Path 5	∞	2	2	1	gen_shift_out[1].um/tmp_reg[1]/C	sum[1]	4.058	3.258	0.800	∞				0.000
Path 6	∞	2	3	2	a[0]	gen_shift_out[0].p_reg[1].sr3/D	2.265	1.132	1.133	∞	input port clock			0.000
Path 7	∞	2	3	2	b[0]	gen_FA[0]FA_i/cout_reg/D	1.906	1.106	0.800	∞	input port clock			0.000
Path 8	∞	2	2	2	gen_shift_in[1].SR_b/tmp_reg[0]/C	gen_shift_out[1].p_reg[0].sr2/D	1.862	0.777	1.085	∞				0.000
Path 9	∞	1	2	1	a[1]	gen_shift_in[1].SR_a/tmp_reg[0]/D	1.782	0.982	0.800	∞	input port clock			0.000
Path 10	∞	1	2	1	b[1]	gen_shift_in[1].SR_b/tmp_reg[0]/D	1.782	0.982	0.800	∞	input port clock			0.000

Από την ανάλυση της προηγούμενης εργαστηριακής άσκησης είχε προκύψει ότι για τον απλό 4-bit Full Adder η συνολική καθυστέρηση του critical path είναι 5.970 ns (από είσοδο σε έξοδο), ενώ για τον σύγχρονο (pipelined) 4-bit αθροιστή η καθυστέρηση του critical path προέκυψε ότι είναι 4.076 ns.

Δεδομένου ότι η περίοδος του ρολογιού καθορίζεται από το critical path, οι μέγιστες συχνότητες λειτουργίας δίνονται από:

$$f_{\max, \text{FA}} = \frac{1}{5.970 \text{ ns}} \approx 167.5 \text{ MHz}$$

$$f_{\max, \text{pipeline}} = \frac{1}{4.076 \text{ ns}} \approx 245.3 \text{ MHz}$$

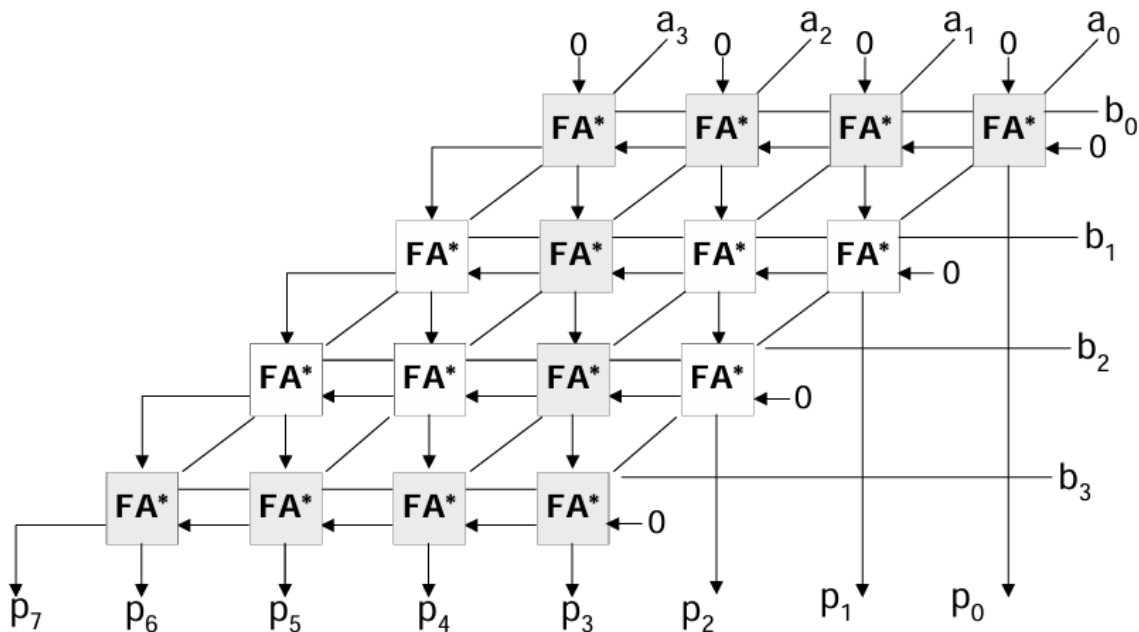
Παρατηρούμε ότι η pipelined υλοποίηση επιτρέπει υψηλότερη συχνότητα λειτουργίας, καθώς το delay του critical path έχει μειωθεί. Από την άλλη όμως, η χρήση pipeline εισάγει latency, καθώς απαιτούνται πολλοί κύκλοι ρολογιού μέχρι να εμφανιστεί το πρώτο αποτέλεσμα (στην περίπτωση μας 4 κύκλοι).

Επομένως, αν το pipeline δεν αξιοποιείται (δηλαδή αν μας ενδιαφέρει η παραγωγή ενός μόνο αποτελέσματος), η απλή υλοποίηση του προηγούμενου εργαστηρίου είναι αποδοτικότερη, καθώς ολοκληρώνει τον υπολογισμό σε έναν κύκλο. Αντίθετα, σε συνθήκες συνεχούς ροής δεδομένων, η pipelined υλοποίηση υπερέχει, αφού μετά την αρχική καθυστέρηση μπορεί να παράγει ένα αποτέλεσμα ανά κύκλο ρολογιού, επιτυγχάνοντας μεγαλύτερο throughput.

Όσον αφορά την κατανάλωση πόρων, η pipelined υλοποίηση απαιτεί περισσότερα στοιχεία υλικού (κυρίως καταχωρητές για την υλοποίηση των pipeline σταδίων), γεγονός που αυξάνει την πολυπλοκότητα και το κόστος του κυκλώματος. Παρατηρούμε δηλαδή ένα αναμενόμενο trade off ανάμεσα σε throughput και κόστος κυκλώματος.

Θέμα 3: Συστολικός Πολλαπλασιαστής διάδοσης κρατουμένου των 4 bits

Η σχεδίαση του συστολικού πολλαπλασιαστή διάδοσης κρατουμένου των 4 bits βασίζεται στο παρακάτω κύκλωμα:



Σημειώνεται ότι, για να χαρακτηριστεί ένα κύκλωμα ως συστολικό (systolic), θα πρέπει να ικανοποιούνται οι ακόλουθες βασικές προϋποθέσεις:

- Τα κύτταρα που συνθέτουν το κύκλωμα είναι πανομοιότυπα και υλοποιούν την ίδια βασική λειτουργία.
- Υποστηρίζεται η συνεχής ροή δεδομένων μεταξύ των κυττάρων.
- Η επικοινωνία μεταξύ των κυττάρων πραγματοποιείται συγχρονισμένα, με παρεμβολή στοιχείων αποθήκευσης (καταχωρητών ή latches), ώστε να εξασφαλίζεται η σταδιακή προώθηση των δεδομένων στο pipeline.

Αρχικά, απαιτείται ο σχεδιασμός της βασικής δομικής μονάδας του πολλαπλασιαστή, η οποία αποτελείται από έναν πλήρη αθροιστή σε συνδυασμό με μία πύλη AND, όπως φαίνεται στο παρακάτω σχήμα. Πιο συγκεκριμένα, το βασικό αυτό κύτταρο είναι υπεύθυνο για τον υπολογισμό του ενδιάμεσου γινομένου δύο bits μέσω της πύλης AND, το οποίο στη συνέχεια αθροίζεται με το αντίστοιχο ενδιάμεσο άθροισμα και το κρατούμενο (carry) της προηγούμενης βαθμίδας μέσω του πλήρους αθροιστή. Επιπλέον, το κύτταρο όπως φαίνεται και παρακάτω περιέχει flip-flops, τα οποία εισάγουν τις απαραίτητες χρονικές καθυστερήσεις, ώστε να εξασφαλίζεται ο σωστός συγχρονισμός των δεδομένων μεταξύ των διαδοχικών σταδίων και η ορθή λειτουργία του pipeline, με τον τρόπο αυτό, τα αποτελέσματα προωθούνται στην επόμενη βαθμίδα με ελεγχόμενη καθυστέρηση, διατηρώντας τη χρονική ευθυγράμμιση του συστήματος.


```

        d  : in  std_logic;
        q  : out std_logic
    );
end component;

signal ab : std_logic;

begin

    ab <= a and b;

    process(clk)
    begin
        if(rising_edge(clk)) then
            sum <= ab xor sin xor cin;
            cout <= (ab and sin) or (ab and cin) or (sin and cin);
        end if;
    end process;

    SR_a: shift_reg
        generic map (DELAY => 2)
        port map (
            clk => clk,
            d  => a,
            q  => a_delayed
        );

    SR_b: shift_reg
        generic map (DELAY => 1)
        port map (
            clk => clk,
            d  => b,
            q  => b_delayed
        );

end Behavioral;

```

Ο κώδικας σε vhdl για την περιγραφή του πολλαπλασιαστή και η structural αρχιτεκτονική του φαίνονται παρακάτω:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Syst_4Mult is
    Port (
        a, b : in  std_logic_vector(3 downto 0);
        p     : out std_logic_vector(7 downto 0);
        clk  : in  std_logic
    );
end Syst_4Mult;

architecture Behavioral of Syst_4Mult is

    component Sync_FA_MULT is
        Port (

```

```

        -- FA
        a, b, cin, sin : in  std_logic;
        cout, sum      : out std_logic;
        a_delayed, b_delayed : out std_logic;

        -- FF
        clk : in std_logic
    );
end component;

component shift_reg is
    generic (DELAY : integer := 1);
    Port (
        clk : in  std_logic;
        d   : in  std_logic;
        q   : out std_logic
    );
end component;

type matrix_5x5 is array (0 to 4) of std_logic_vector(4 downto 0);
signal sout_wire, cout_wire : matrix_5x5;
signal a_wire, b_wire      : matrix_5x5;
signal cout_del            : std_logic_vector(2 downto 0);
signal p_zero_to_two      : std_logic_vector(3 downto 0);

begin

    a_wire(0)(0) <= a(0);
    b_wire(0)(0) <= b(0);

    gen_rows: for i in 0 to 3 generate
        gen_cols: for j in 0 to 3 generate

            begin

                first_cell: if (i = 0 and j = 0) generate
                    Sync_FA_MULT_inst: Sync_FA_MULT
                        port map (
                            a  => a_wire(i)(j),
                            b  => b_wire(i)(j),
                            clk => clk,

                            cin => '0',
                            sin => '0',
                            sum => p_zero_to_two(i),

                            cout      => cout_wire(i)(j+1),
                            a_delayed => a_wire(i+1)(j),
                            b_delayed => b_wire(i)(j+1)
                        );
                end generate;

                top_row: if (i = 0 and j > 0) generate
                    Sync_FA_MULT_inst: Sync_FA_MULT
                        port map (

```

```

        a  => a_wire(i)(j),
        b  => b_wire(i)(j),
        cin => cout_wire(i)(j),
        sin => '0',
        clk => clk,

        cout      => cout_wire(i)(j+1),
        sum       => sout_wire(i+1)(j-1),
        a_delayed => a_wire(i+1)(j),
        b_delayed => b_wire(i)(j+1)
    );
end generate;

left_column: if (i > 0 and j = 3) generate
    Sync_FA_MULT_inst: Sync_FA_MULT
        port map (
            a  => a_wire(i)(j),
            b  => b_wire(i)(j),
            clk => clk,

            cin => cout_wire(i)(j),
            sin => cout_del(i-1),
            sum => sout_wire(i+1)(j-1),

            cout      => cout_wire(i)(j+1),
            a_delayed => a_wire(i+1)(j),
            b_delayed => b_wire(i)(j+1)
        );
end generate;

inner_cells: if (i > 0 and (j > 0 and j < 3)) generate
    Sync_FA_MULT_inst: Sync_FA_MULT
        port map (
            a  => a_wire(i)(j),
            b  => b_wire(i)(j),
            clk => clk,

            cin => cout_wire(i)(j),
            sin => sout_wire(i)(j),
            sum => sout_wire(i+1)(j-1),

            cout      => cout_wire(i)(j+1),
            a_delayed => a_wire(i+1)(j),
            b_delayed => b_wire(i)(j+1)
        );
end generate;

right_column: if (i > 0 and j = 0) generate
    Sync_FA_MULT_inst: Sync_FA_MULT
        port map (
            a  => a_wire(i)(j),
            b  => b_wire(i)(j),
            clk => clk,

            cin => '0',

```

```

        sin => sout_wire(i)(j),
        sum => p_zero_to_two(i),

        cout      => cout_wire(i)(j+1),
        a_delayed => a_wire(i+1)(j),
        b_delayed => b_wire(i)(j+1)
    );
    end generate;

    end generate;
end generate;

gen_shift_in_a: for i in 0 to 2 generate
    SR_a: shift_reg
        generic map (DELAY => i+1)
        port map (
            clk => clk,
            d   => a(i+1),
            q   => a_wire(0)(i+1)
        );
end generate;

gen_shift_out_3_to_7: for i in 0 to 1 generate
    SR_p_1: shift_reg
        generic map (DELAY => 2-i)
        port map (
            clk => clk,
            d   => sout_wire(4)(i),
            q   => p(4+i)
        );
end generate;

gen_shift_out_0_to_3: for i in 0 to 3 generate
    SR_p_2: shift_reg
        generic map (DELAY => 3 + 2*i)
        port map (
            clk => clk,
            d   => p_zero_to_two(3-i),
            q   => p(3-i)
        );
end generate;

gen_shift_cout: for i in 0 to 2 generate
    SR_cout: shift_reg
        generic map (DELAY => 1)
        port map (
            clk => clk,
            d   => cout_wire(i)(4),
            q   => cout_del(i)
        );
end generate;

gen_shift_in_b: for i in 0 to 2 generate
    SR_bt: shift_reg
        generic map (DELAY => 2*(i+1))

```

```

port map (
    clk => clk,
    d   => b(i+1),
    q   => b_wire(i+1)(0)
);
end generate;

P(6) <= sout_wire(4)(2);
p(7) <= cout_wire(3)(4);

end Behavioral;

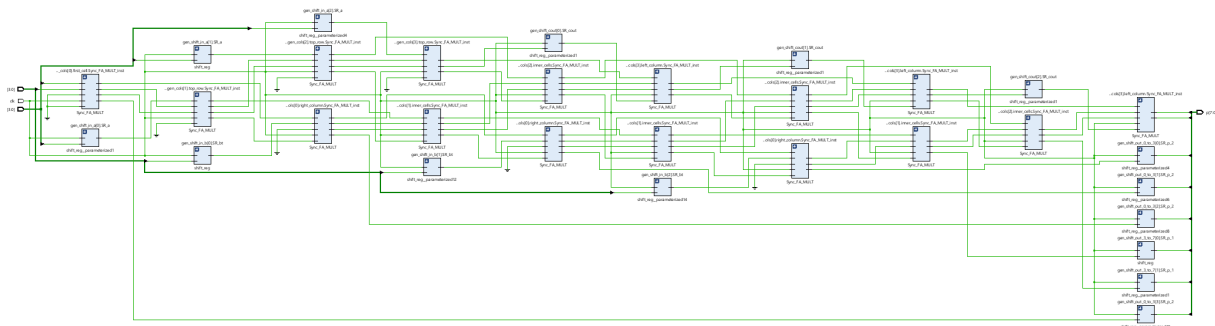
```

Ο παραπάνω πολλαπλασιαστής αποτελείται από 16 full adders * χωρισμένους σε 10 επίπεδα. Επιπλέον, έχουν σχεδιαστεί οι απαραίτητοι registers για τις καθυστερήσεις που χρειάζονται στην είσοδο και στην έξοδο του πολλαπλασιαστή. Όπως έχουμε αναφέρει και παραπάνω, οι ενδιάμεσες καθυστερήσεις ανάμεσα στους full adders * σχεδιάζονται μέσα στα βασικά κύτταρα full adders *.

(Σημειώνεται ότι τέσσερις από τους συνολικά 16 full adders θα μπορούσαν να παραλειφθούν, καθώς στην πρώτη σειρά κάθε full adder* αξιοποιεί ουσιαστικά μόνο τη λειτουργία της πύλης AND, αφού το ένα από τα δύο εισερχόμενα bits στον full adder του full adder* είναι σταθερά 0. Με αυτή τη βελτιστοποίηση θα μπορούσε να μειωθεί και το latency του pipeline, εξοικονομώντας επιπλέον κύκλους.

Ωστόσο, επιλέχθηκε η χρήση ομοιόμορφων δομικών μονάδων σε όλο το κύκλωμα, προκειμένου να διατηρηθεί η κανονικότητα της αρχιτεκτονικής, που αποτελεί βασική λογική στα systolic designs. Επιπλέον, η ελαχιστοποίηση των pipeline cycles δεν αποτελούσε βασικό ζητούμενο της συγκεκριμένης υλοποίησης, με αποτέλεσμα να προτιμηθεί η παραπάνω, πιο ομοιόμορφη σχεδίαση.)

Με χρήση του elaborate design έχουμε:



Για τον έλεγχο της ορθής λειτουργίας του πολλαπλασιαστή χρησιμοποιείται το παρακάτω testbench. Αρχικά, εκτελούνται ορισμένοι ενδεικτικοί πολλαπλασιασμοί, ενώ στη συνέχεια, το testbench επεκτείνεται σε εξαντλητικό έλεγχο, δοκιμάζοντας όλους τους δυνατούς συνδυασμούς εισόδων. Σημειώνεται ότι προφανώς στην πράξη δεν εκτελέστηκε προσομοίωση μέχρι την πλήρη ολοκλήρωση όλων των περιπτώσεων, ωστόσο θεωρητικά, αν δίναμε τον απαραίτητο χρόνο, το testbench θα κάλυπτε το σύνολο των πιθανών συνδυασμών.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity fa_mul_tb is
end fa_mul_tb;

architecture tb of fa_mul_tb is

    component Syst_4Mult
        Port (
            a, b : in  std_logic_vector(3 downto 0);
            p    : out std_logic_vector(7 downto 0);
            clk  : in  std_logic
        );
    end component;

    signal a    : std_logic_vector(3 downto 0) := (others => '0');
    signal b    : std_logic_vector(3 downto 0) := (others => '0');
    signal p    : std_logic_vector(7 downto 0);
    signal clk  : std_logic := '0';

    constant CLOCK_PERIOD : time := 10 ns;

begin

    UUT: Syst_4Mult
        port map (
            a => a,
            b => b,
            p => p,
            clk => clk
        );

    CLK_GEN : process
    begin
        clk <= '0';
        wait for CLOCK_PERIOD / 2;
        clk <= '1';
        wait for CLOCK_PERIOD / 2;
    end process;

    STIMULUS : process
    begin
        -- Test case 1: 3 * 2 = 6
        a <= "0011";
        b <= "0010";
        wait for CLOCK_PERIOD;
```

```

-- Test case 2: 7 * 5 = 35
a <= "0111";
b <= "0101";
wait for CLOCK_PERIOD;

-- Test case 3: 15 * 15 = 225
a <= "1111";
b <= "1111";
wait for CLOCK_PERIOD;

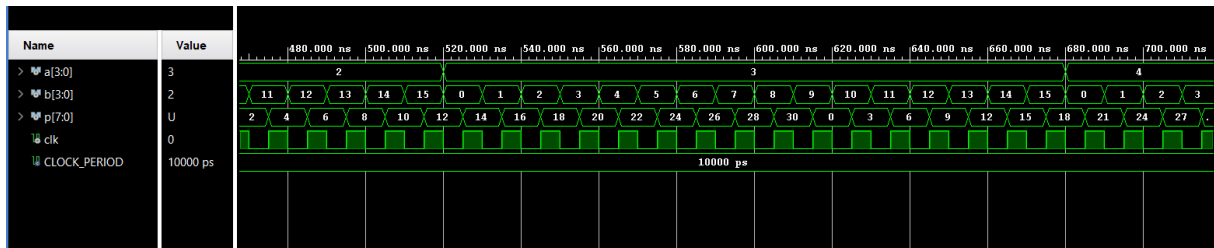
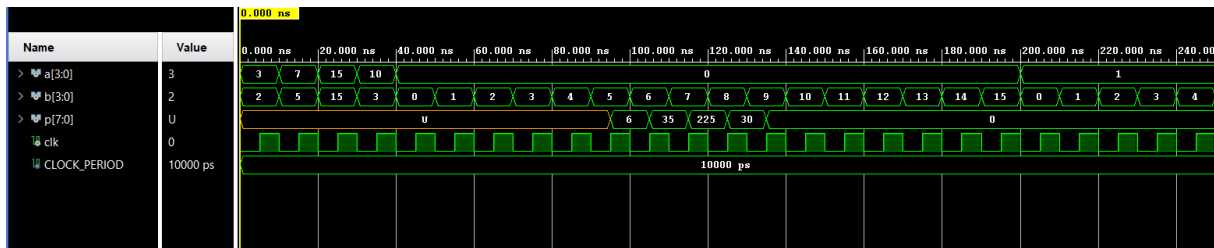
-- Test case 4: 10 * 3 = 30
a <= "1010";
b <= "0011";
wait for CLOCK_PERIOD;

for i in 0 to 15 loop
    a <= std_logic_vector(to_unsigned(i, 4));
    for j in 0 to 15 loop
        b <= std_logic_vector(to_unsigned(j, 4));
        wait for 1* CLOCK_PERIOD;
    end loop;
end loop;

wait;
end process;

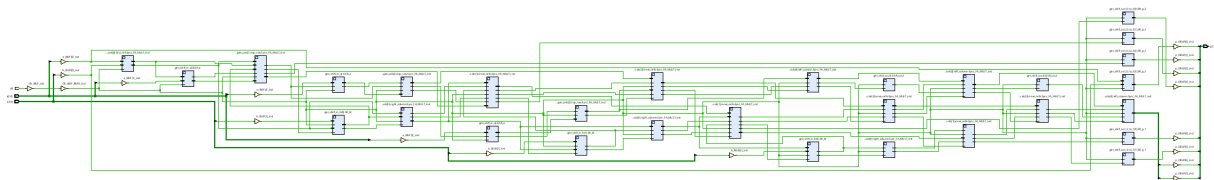
end tb;

```



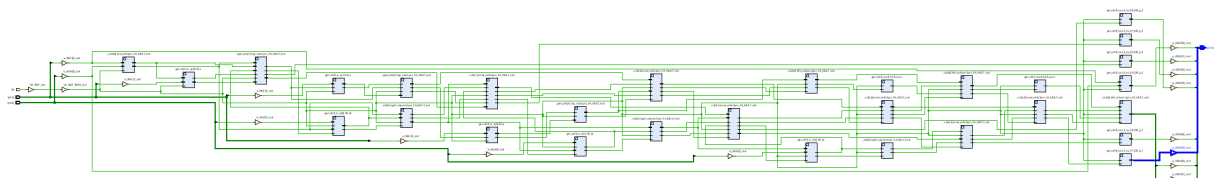
Στις παραπάνω εικόνες δεν είναι δυνατό να αποτυπωθεί το σύνολο των αποτελεσμάτων που παράγαμε από την προσμείωση. Παρόλα αυτά, από τη συνολική συμπεριφορά που παρατηρείται ακόμα και από αυτά τα δύο στιγμιότυπα που παραθέσαμε γίνεται σαφές ότι το κύκλωμα λειτουργεί ορθά. Επιπλέον, διακρίνεται η αρχική καθυστέρηση του συστήματος, λόγω του pipeline, οπου μετά το πέρας της οποίας το κύκλωμα αρχίζει να παράγει αποτελέσματα σε κάθε κύκλο ρολογιού, όπως αναμένεται.

Τέλος με χρήση του synthesis έχουμε:



Για να βρούμε το critical path του συνολικού μας κυκλώματος και την αντίστοιχη χρονική καθυστέρηση με χρήση του timing summary έχουμε:

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
Path 1	∞	2	2	1	gen_shift_out_3_1/tmp_reg[0]/C	pl[5]	4.076	3.276	0.800	∞				0.000
Path 2	∞	2	2	1	gen_rows[3].gen_inst/sum_reg/C	pl[6]	4.076	3.276	0.800	∞				0.000
Path 3	∞	2	2	1	gen_rows[3].gen_inst/cout_reg/C	pl[7]	4.076	3.276	0.800	∞				0.000
Path 4	∞	2	2	1	gen_shift_out_0_2/tmp_reg[8]/C	pl[0]	4.058	3.258	0.800	∞				0.000
Path 5	∞	2	2	1	gen_shift_out_0_2/tmp_reg[6]/C	pl[1]	4.058	3.258	0.800	∞				0.000
Path 6	∞	2	2	1	gen_shift_out_0_2/tmp_reg[4]/C	pl[2]	4.058	3.258	0.800	∞				0.000
Path 7	∞	2	2	1	gen_shift_out_0_2/tmp_reg[2]/C	pl[3]	4.058	3.258	0.800	∞				0.000
Path 8	∞	2	2	1	gen_shift_out_3_1/tmp_reg[1]/C	pl[4]	4.058	3.258	0.800	∞				0.000
Path 9	∞	2	3	2	b[0]	gen_shift_out_0_2/mp_reg[7].sr[9]/D	2.239	1.106	1.133	∞	input port clock			0.000
Path 10	∞	2	2	2	gen_rows[2].gen_a/tmp_reg[1]/C	gen_shift_out_3_1/mp_reg[0].sr[2]/D	1.862	0.777	1.085	∞				0.000



Παρατηρούμε λοιπόν ότι το critical path μας έχει συνολική καθυστέρηση 4.076ns

Θέμα bonus: Πέρασμά κώδικα στην πλακέτα zybo και πειραματισμός με αυτόν

Για την επιτυχή ολοκλήρωση αυτής της αρχικής επαφής με την πλακέτα Zybo, ακολουθήσαμε τα βήματα των οδηγιών και επαληθεύσαμε τη σωστή λειτουργία του συστήματος παρατηρώντας ένα LED να αναβοσβήνει.

Στη συνέχεια, τροποποιήσαμε τον δοσμένο κώδικα, προκειμένου να κατανοήσουμε λίγο καλύτερα τη λειτουργία του. Πιο συγκεκριμένα :

(ο κώδικας σε αυτό το σημείο παρατίθεται για να είναι πιο ευανάγνωστη η αναφορά, αφού στη συνέχεια κάνουμε reference σε μεταβλητές του κώδικα)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use ieee.std_logic_unsigned.all;

entity blink_led is
    generic (
        COUNTER_LIMIT : natural := 28
    );
    port (
        -----
        -- CONTROL SIGNALS
        i_clk   : in    std_logic;
        -----
        -- MAIN SIGNALS
        o_led0  : out   std_logic
    );
end entity;

architecture behavioral_arch of blink_led is

    signal count_reg : std_logic_vector(COUNTER_LIMIT-1 downto 0) := (others => '0');

begin

    BLINK_LED_PROCESS : process(i_clk)
    begin
        if (rising_edge(i_clk)) then
            count_reg <= count_reg + 1;
        end if;
    end process;

    o_led0 <= count_reg(COUNTER_LIMIT-1);

end behavioral_arch;
```

Παρατηρήσαμε ότι η τιμή της μεταβλητής COUNTER_LIMIT επηρεάζει άμεσα τη συχνότητα με την οποία αναβοσβήνει το LED. Συγκεκριμένα, το LED είναι αναμμένο όταν το MSB του count_reg είναι 1 και σβηστό όταν είναι 0, ενώ το μήκος του count_reg καθορίζεται από την τιμή του COUNTER_LIMIT.

Κατά συνέπεια, όσο αυξάναμε την τιμή της COUNTER_LIMIT, το LED αναβόσβηνε πιο αργά, ενώ όσο τη μειώναμε, αναβόσβηνε πιο γρήγορα. Μάλιστα, κάτω από μία ορισμένη τιμή, η εναλλαγή γινόταν τόσο γρήγορα ώστε το LED να φαίνεται στο ανθρώπινο μάτι σαν να παραμένει συνεχώς αναμμένο.

Επιπλέον, τροποποιώντας ελαφρώς τον αρχικό κώδικα, καταφέραμε να πετύχουμε και duty cycle διαφορετικό από 50%. Αντί δηλαδή το LED να παραμένει ίσο χρόνο αναμμένο και σβηστό, μπορέσαμε να ορίσουμε διαφορετική διάρκεια για τις δύο αυτές καταστάσεις, συγκρίνοντας την τιμή του μετρητή με ένα κατάλληλο (threshold), όπως φαίνεται παρακάτω:

```
o_led0 <= '1' when count_reg < threshold else '0';
```

Με τον τρόπο αυτό, το LED παραμένει αναμμένο μόνο για ένα επιλεγμένο τμήμα της συνολικής περιόδου λειτουργίας του μετρητή, γεγονός που επιτρέπει την υλοποίηση duty cycle διαφορετικού του 50%.

Ο πλήρης τροποποιημένος κώδικας παρουσιάζεται στη συνέχεια:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity blink_led is
    generic (
        COUNTER_LIMIT : natural := 28
    );
    port (
        -----
        -- CONTROL SIGNALS
        i_clk    : in    std_logic;
        -----
        -- MAIN SIGNALS
        o_led0   : out   std_logic
    );
end entity;

architecture behavioral_arch of blink_led is

    signal count_reg : unsigned(COUNTER_LIMIT-1 downto 0) := (others => '0');
    signal threshold : unsigned(COUNTER_LIMIT-1 downto 0) := to_unsigned(192, COUNTER_LIMIT);

begin

    BLINK_LED_PROCESS : process(i_clk)
    begin
        if (rising_edge(i_clk)) then
            count_reg <= count_reg + 1;
        end if;
    end process;

    o_led0 <= '1' when count_reg < threshold else '0';

end behavioral_arch;
```

Τέλος, μια επιπλέον παρατήρηση που προέκυψε εκ των υστέρων και δυστυχώς δεν δοκιμάστηκε στην πράξη, είναι ότι, τροποποιώντας το αρχείο των constraints, θα μπορούσαμε να οδηγήσουμε επιπλέον LEDs ώστε να αναβοσβήνουν. Συγκεκριμένα, κάνοντας uncomment τις αντίστοιχες γραμμές και ορίζοντας κατάλληλα ονόματα θυρών για τα LEDs που είναι ήδη δηλωμένα αλλά σχολιασμένα κάτω από το ενεργό LED, θα ήταν δυνατόν, με την απαραίτητη τροποποίηση και του κώδικα στα design sources, να οδηγήσουμε περισσότερα LEDs.

```
# Clock signal
set_property -dict { PACKAGE_PIN L16    IOSTANDARD LVCMOS33 } [get_ports { i_clk }];
create_clock -add -name sys_clk_pin -period 8.00 -waveform {0 4} [get_ports { i_clk }];

# LEDs
set_property -dict { PACKAGE_PIN M14    IOSTANDARD LVCMOS33 } [get_ports { o_led0 }];
#set_property -dict { PACKAGE_PIN M15    IOSTANDARD LVCMOS33 } [get_ports { led[1] }];
```

```
#set_property -dict { PACKAGE_PIN G14    IOSTANDARD LVCMOS33 } [get_ports { led[2] }];  
#set_property -dict { PACKAGE_PIN D18    IOSTANDARD LVCMOS33 } [get_ports { led[3] }];
```